



UET.CS2043 - Lập trình nâng cao

Mô hình MVC

Ứng dụng quá lớn

KHÓ

bảo trì
tái sử dụng
làm việc nhóm

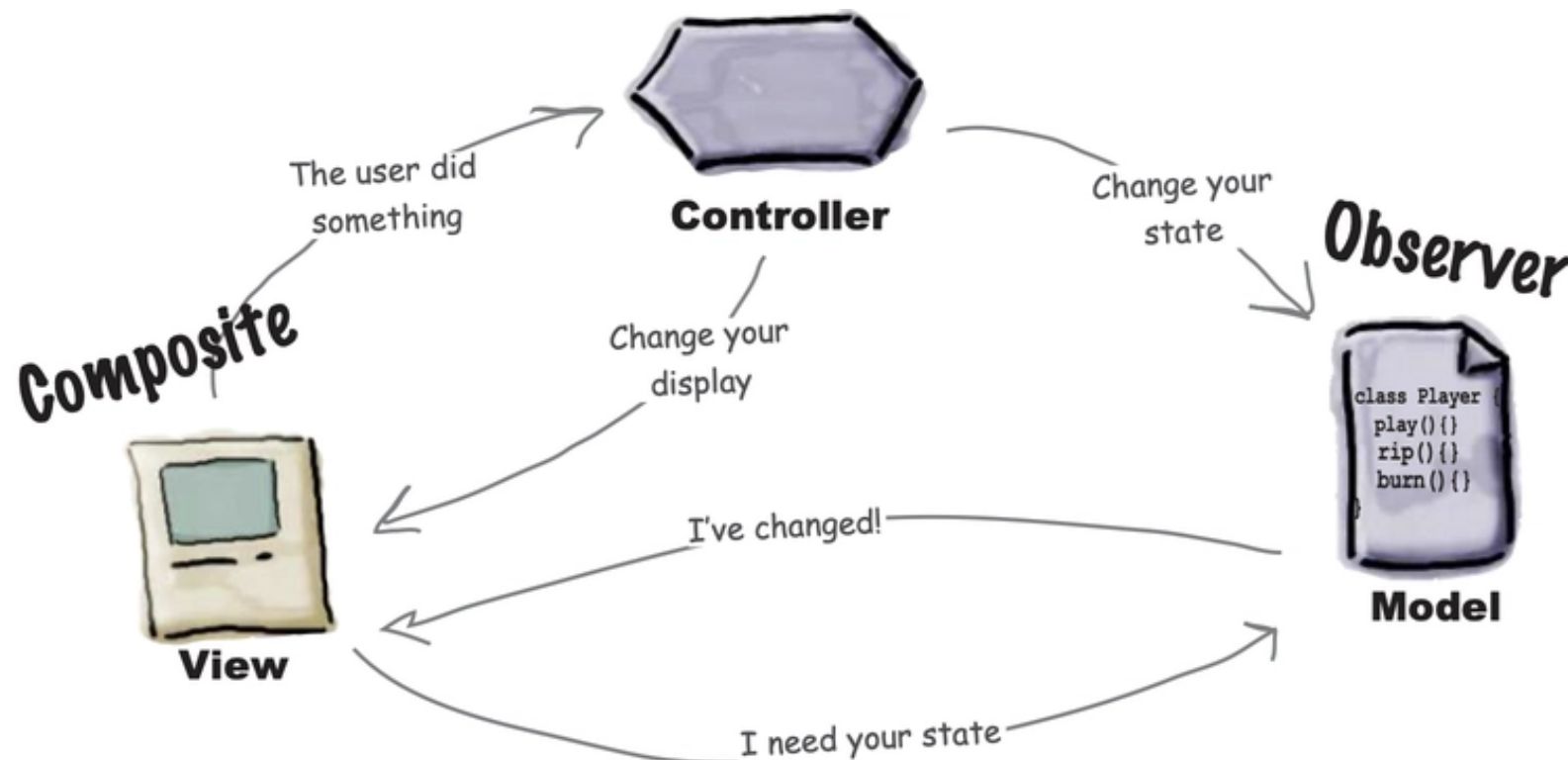
 **Application.java**

```
9996      try
9997      {
9998          String url="jdbc:oracle
9999          String user="system";
```

MVC (Model - View - Controller)

Giải pháp: “chia để trị” với MVC

MVC mà một trong những mẫu kiến trúc phổ biến nhất cho ứng dụng Web.



MVC (Model - View - Controller)

Lợi ích:

- Tách biệt mối quan tâm về nghiệp vụ cho các thành phần
- Tăng khả năng bảo trì và mở rộng
- Song song hóa quá trình phát triển
- Tái sử dụng mã nguồn

Các thành phần trong MVC

- **Model:**

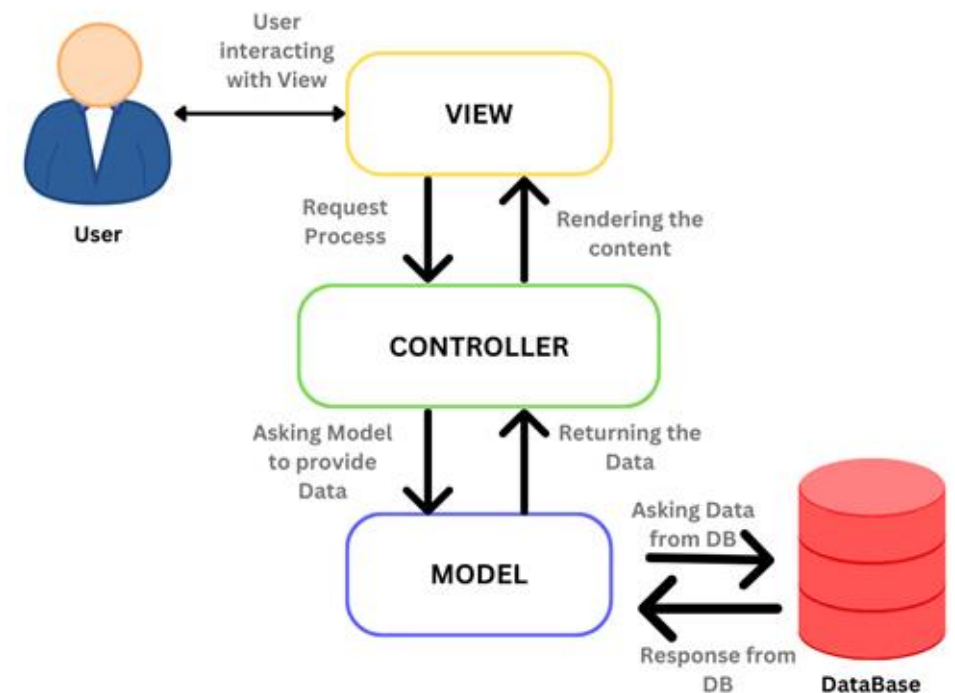
- Các lớp đối tượng liên quan đến dữ liệu
- Phần logic tương tác đến lưu trữ dữ liệu

- **View:**

- Hiển thị giao diện đến người dùng
- Xử lý việc hiển thị dữ liệu

- **Controller:**

- Xử lý các tương tác với View và Model



Ví dụ: MVC

```
uet.dse
├── MainApp.java          # Lớp khởi tạo Scene và nạp Login View
├── database              # Tầng kết nối dữ liệu
│   └── DatabaseConnection.java # Quản lý Connection (JDBC)
├── models                # Tầng dữ liệu (Dữ liệu thô)
│   └── User.java         # Entity User (username, hashPassword)
├── controllers           # Tầng điều khiển (Xử lý sự kiện UI)
│   └── LoginController.java # Bắt sự kiện click nút Login, đọc thông tin đăng nhập
└── resources            # Thư mục chứa tài nguyên (ngoài src/main/java)
    ├── uet/dse/
    │   ├── views/
    │   │   ├── login-view.fxml # Giao diện XML
    │   │   └── css/
    │   │       └── login-style.css # Định dạng giao diện
```

Ví dụ: MVC

```
//User.java
public class User {
    public String username;
    public String password;

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }
}
```

Dữ liệu (Model)

```
<!-- login-view.fxml -->
<VBox alignment="CENTER" spacing="10" xmlns:fx="http://javafx.com/fxml/1"
    fx:controller="uet.dse.controllers.LoginController">

    <TextField fx:id="txtUser" promptText="Username"/>
    <PasswordField fx:id="txtPass" promptText="Password"/>
    <Button text="Login" onAction="#handleLogin"/>
    <Label fx:id="lblMsg"/>

</VBox>
```

Giao diện (View)

```
//LoginController.java
public class LoginController {
    @FXML private TextField txtUser;
    @FXML private PasswordField txtPass;
    @FXML private Label lblMsg;

    private LoginService service = new LoginService();

    @FXML
    private void handleLogin() {
        if (service.authenticate(txtUser.getText(), txtPass.getText())) {
            lblMsg.setText("Thành công!");
        } else {
            lblMsg.setText("Thất bại.");
        }
    }
}
```

Tương tác với giao diện, gọi đến các services (Controller)

Giao diện đồ họa (GUI) với JavaFX [View]

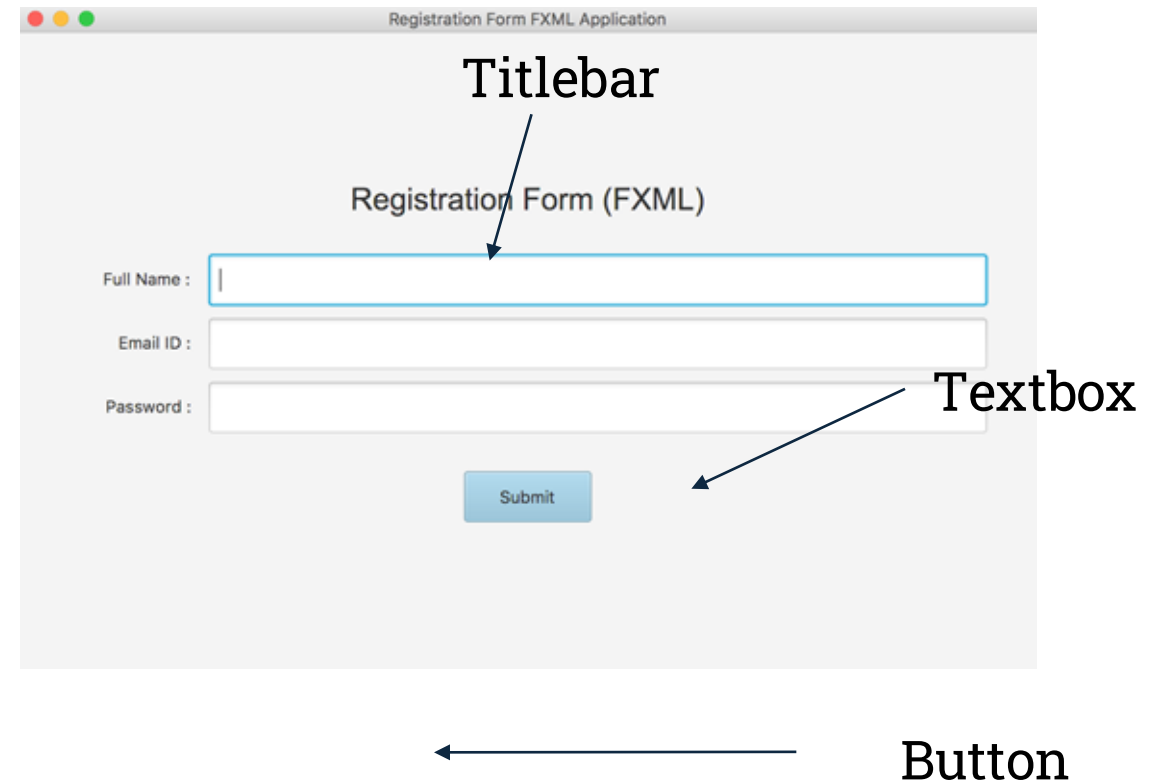
Giao diện đồ hoạ người dùng

Graphical user interface - GUI

Tạo các ứng dụng có giao diện đồ hoạ với các thành phần tương tác trực tiếp với người dùng:

- Button
- Textbox, Label
- Checkbox
- List

...



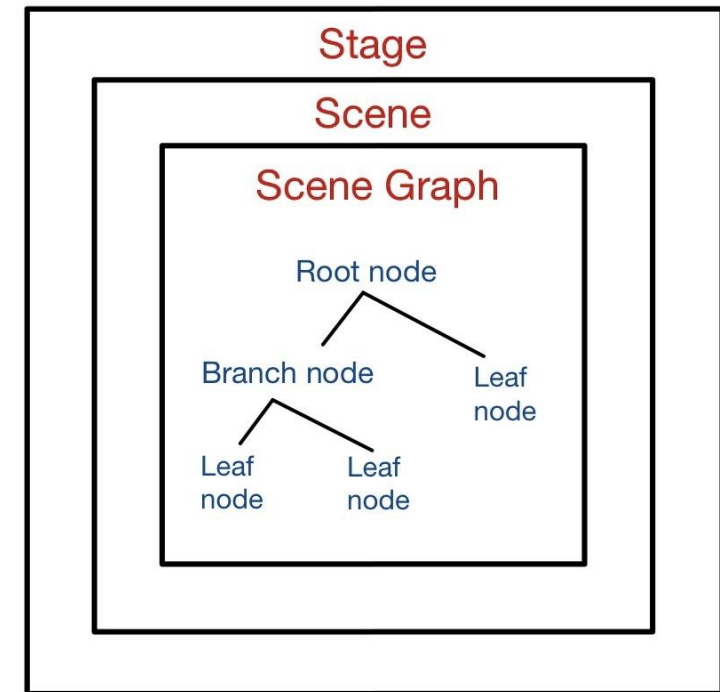
JavaFX

- JavaFX là thư viện đồ hoạ hướng đối tượng
- JavaFX được đi kèm với Oracle JDK cho tới phiên bản 10
- Các lựa chọn thay thế khác: Java Swing, Apache Pivot, SWT

- Hỗ trợ FXML (tương tự HTML)
- Scene Builder: JavaFX cung cấp ứng dụng Scene Builder cho phép kéo thả khi thiết kế
- Tương thích với Swing: có thể nhúng các thành phần Swing
- Built-in UI controls: cung cấp đa dạng controls
- CSS like styling: thiết kế giao diện với tính năng tương tự trong CSS

Cấu trúc của ứng dụng JavaFX

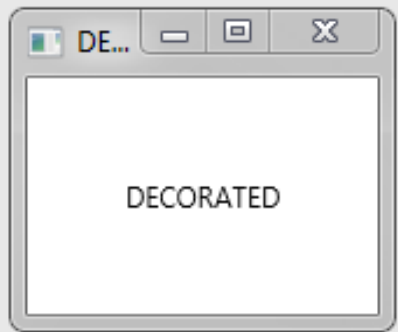
- Một ứng dụng JavaFX được kế thừa từ lớp `javafx.application.Application`
- Triển khai phương thức `start(Stage stage)`
- **Stage** thể hiện cửa sổ ứng dụng.
- Bên trong **Stage** chứa **Scene** và các thành phần con trong **Scene graph**.



Stage

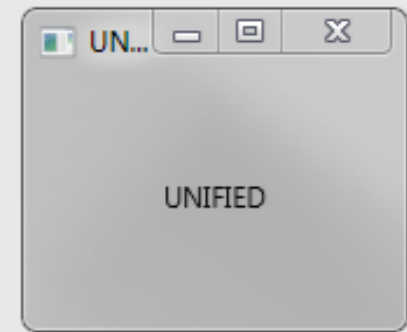
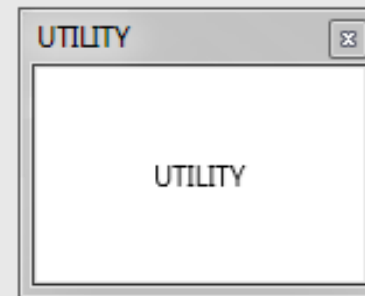
Có 5 styles cho Stage:

1. `stage.initStyle(StageStyle.DECORATED);`
2. `stage.intStyle(StageStyle.UNDECORATED);`
3. `stage.initStyle(StageStyle.TRANSPARENT);`
4. `stage.initStyle(StageStyle.UNIFIED);`
5. `stage.initStyle(StageStyle.UTILITY);`



UNDECORATED

TRANSPARENT



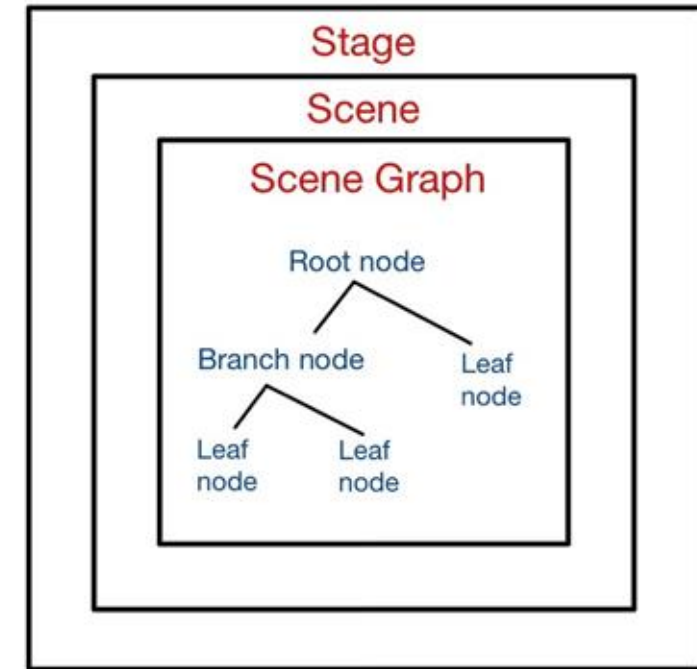
Scene Graph

Scene Graph:

- Cấu trúc dữ liệu phân cấp cây
- Biểu diễn nội dung một Scene: **controls** và **layout**

Node:

- Là lớp cơ sở của các loại Node: [javafx.scene.Node](#)
- Biểu diễn đối tượng đồ họa của một **Scene graph**
 - Các phần tử hình học 2D hoặc 3D: Circle, Rectangle, Polygon, ...
 - Phần tử tương tác: Button, Checkbox, TextArea, ...
 - Phần tử đa phương tiện: Audio, Video, Image, ...

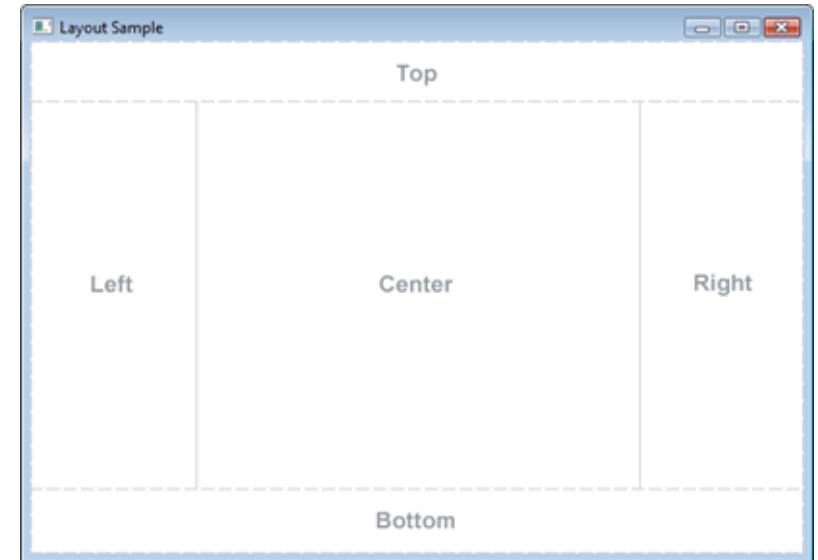


Bố cục với Pane

Được sử dụng để sắp xếp các node trong scene graph.

Mỗi layout là một class kế thừa lớp **Pane** trong gói **javafx.layout**

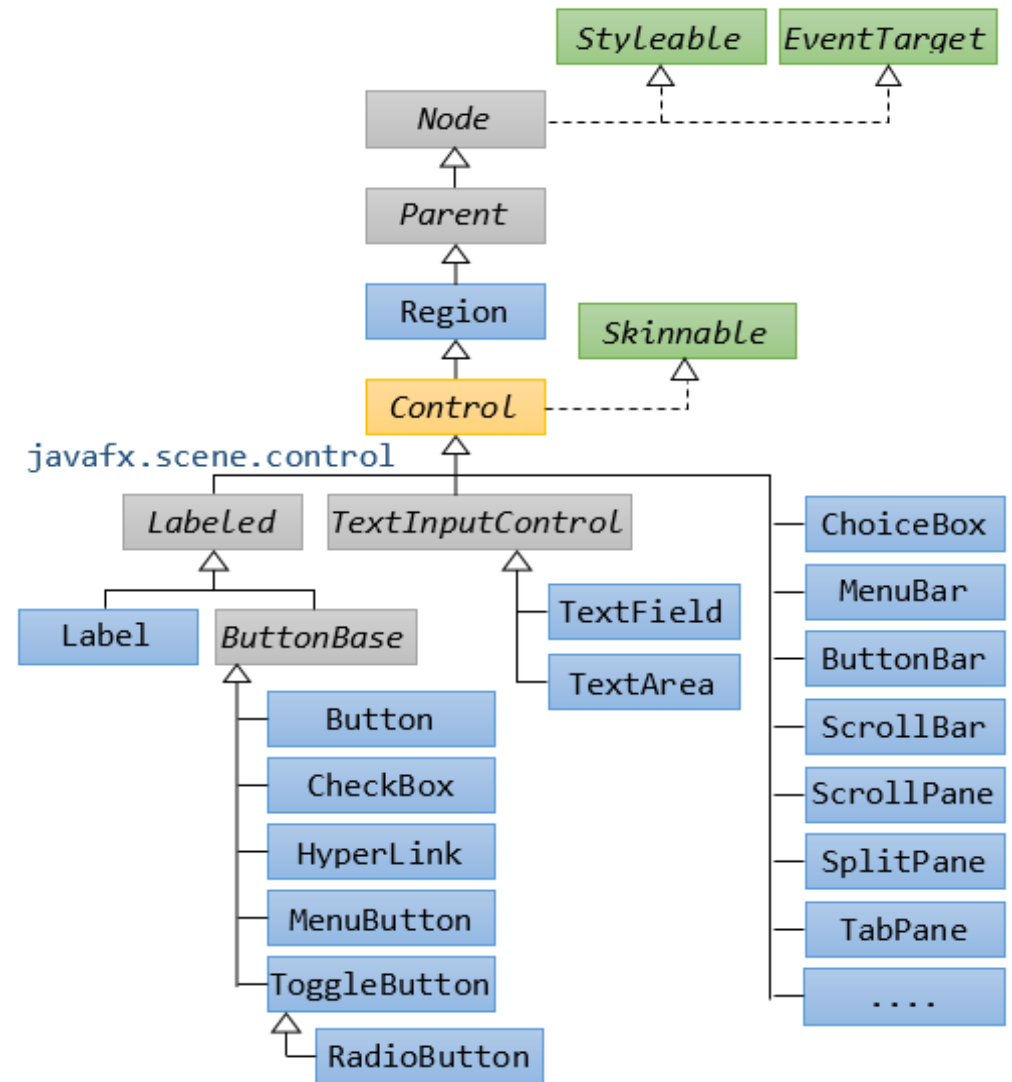
- Border Pane: gồm 5 vùng trên, dưới, trái, phải và chính giữa
- HBox/VBox: tự động căn các đối tượng con theo chiều ngang/dọc.
- Stack Pane: nhiều đối tượng con trên cùng một vị trí (xếp chồng)
- Grid Pane: bố cục dạng lưới
- Flow Pane: bố cục tự điều chỉnh
- AnchorPane: neo các đối tượng tại vị trí cố định



Border Pane

https://docs.oracle.com/javafx/2/layout/builtin_layouts.htm

Built-in UI controls



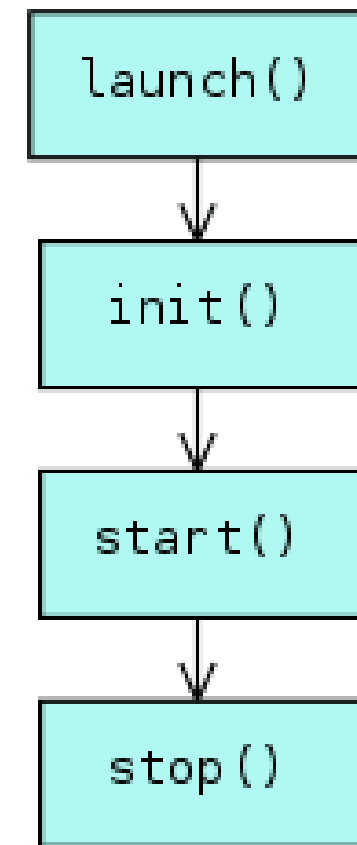
https://docs.oracle.com/javase/8/javafx/user-interface-tutorial/ui_controls.htm

Vòng đời ứng dụng JavaFX

Bắt đầu khi một instance của một lớp thừa kế lớp `Application`

Khi gọi phương thức `A.launch()`:

1. Tạo một instance của lớp A (A kế thừa lớp `Application`)
2. Gọi phương thức `init()`
3. Gọi phương thức `start()` với đối số là một stage
4. Ứng dụng kết thúc khi:
 1. Đóng cửa sổ ứng dụng
 2. Ứng dụng gọi `Platform.exit()` (Không dùng `System.exit()`)
5. Gọi phương thức `stop()`



Ví dụ về ứng dụng JavaFX



```
public class HelloWorld extends Application {  
  
    @Override  
    public void start(Stage stage) {  
        Label label = new Label("Hello World!");  
        label.setFont(new Font(40));  
        Scene scene = new Scene(label);  
  
        stage.setTitle("Welcome to JavaFX!");  
        stage.setScene(scene);  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        Application.launch(args);  
    }  
}
```

← **Hiển thị cửa sổ**

FXML

FXML là ngôn ngữ **XML-based** được tạo ra bởi Oracle Corporation để định nghĩa giao diện cho JavaFX, tương tự HTML cho nền tảng Web.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import java.net.*?>
4  <?import javafx.geometry.*?>
5  <?import javafx.scene.control.*?>
6  <?import javafx.scene.layout.*?>
7  <?import javafx.scene.text.*?>
8
9  <GridPane fx:controller="fxmlexample.FXMLExampleController"
10     xmlns:fx="http://javafx.com/fxml" alignment="center" hgap="10" vgap="10"
11     styleClass="root">
12     <padding><Insets top="25" right="25" bottom="25" left="25"/></padding>
13
14     <Text id="welcome-text" text="Welcome"
15         GridPane.columnIndex="0" GridPane.rowIndex="0"
16         GridPane.columnSpan="2"/>
17
18     <Label text="Password:"
19         GridPane.columnIndex="0" GridPane.rowIndex="2"/>
20
21     <HBox spacing="10" alignment="bottom_right"
22         GridPane.columnIndex="1" GridPane.rowIndex="4">
23         <Button text="Sign In"
24             onAction="#handleSubmitButtonAction"/>
25     </HBox>
26
27 </GridPane>
```

```
@Override
public void start(Stage primaryStage) throws Exception {
    Parent root = FXMLLoader.load(getClass().getResource("main.fxml"));

    primaryStage.setScene(new Scene(root));
    primaryStage.show();
}
```

Cascading Style Sheets (CSS)

- JavaFX Cascading Style Sheets (CSS) được dựa trên W3C CSS, tương tự với CSS for HTML.
- CSS bao gồm **selector** và **declaration block**.
- Dùng để tùy chỉnh màu sắc, text, hiệu ứng, kích thước, v.v.

```
scene.getStylesheets().add("path/stylesheet.css");  
  
Rectangle rect = new Rectangle(100,100);  
rect.getStyleClass().add("my-rect");
```



```
.my-rect {  
    -fx-fill: yellow;  
    -fx-stroke: green;  
    -fx-stroke-width: 5;  
    -fx-stroke-dash-array: 12 2 4 2;  
    -fx-stroke-dash-offset: 6;  
    -fx-stroke-line-cap: butt;  
}
```

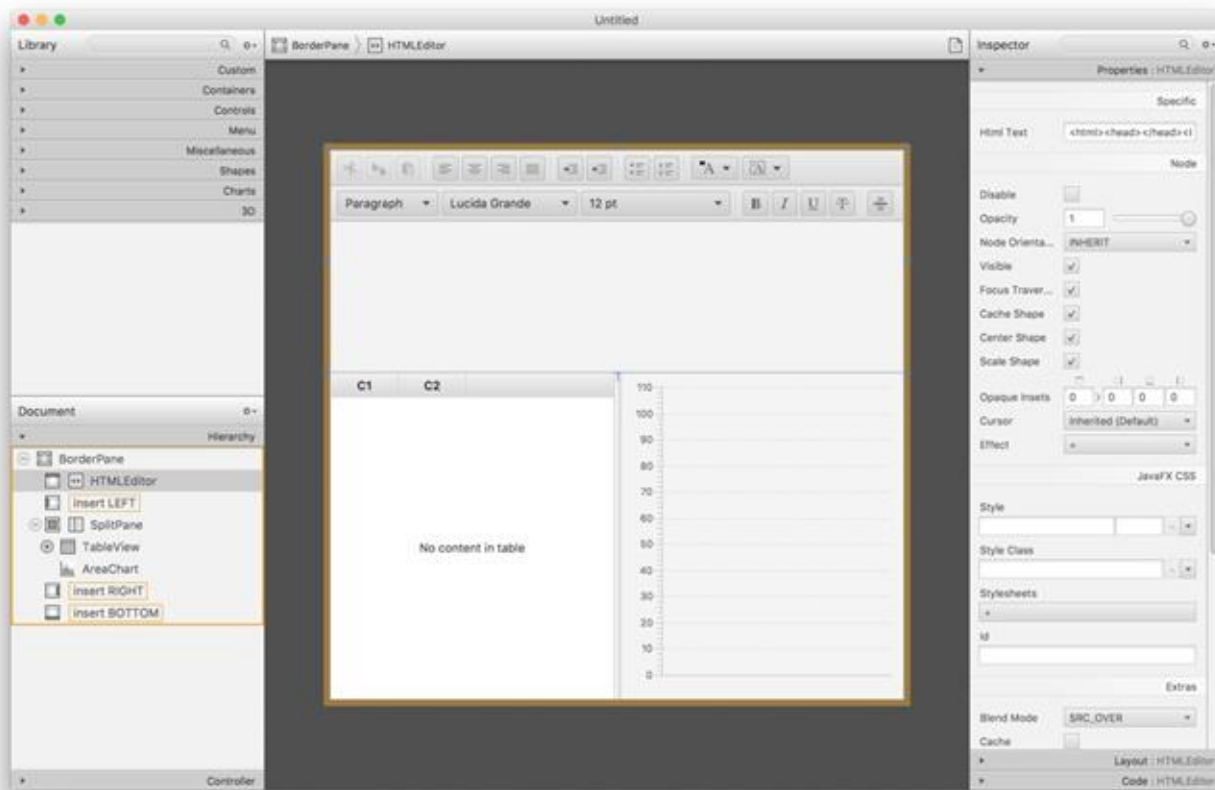
Inline CSS

- Sử dụng qua phương thức **setStyle** của đối tượng.
- Nhanh chóng thêm thuộc tính style mới.

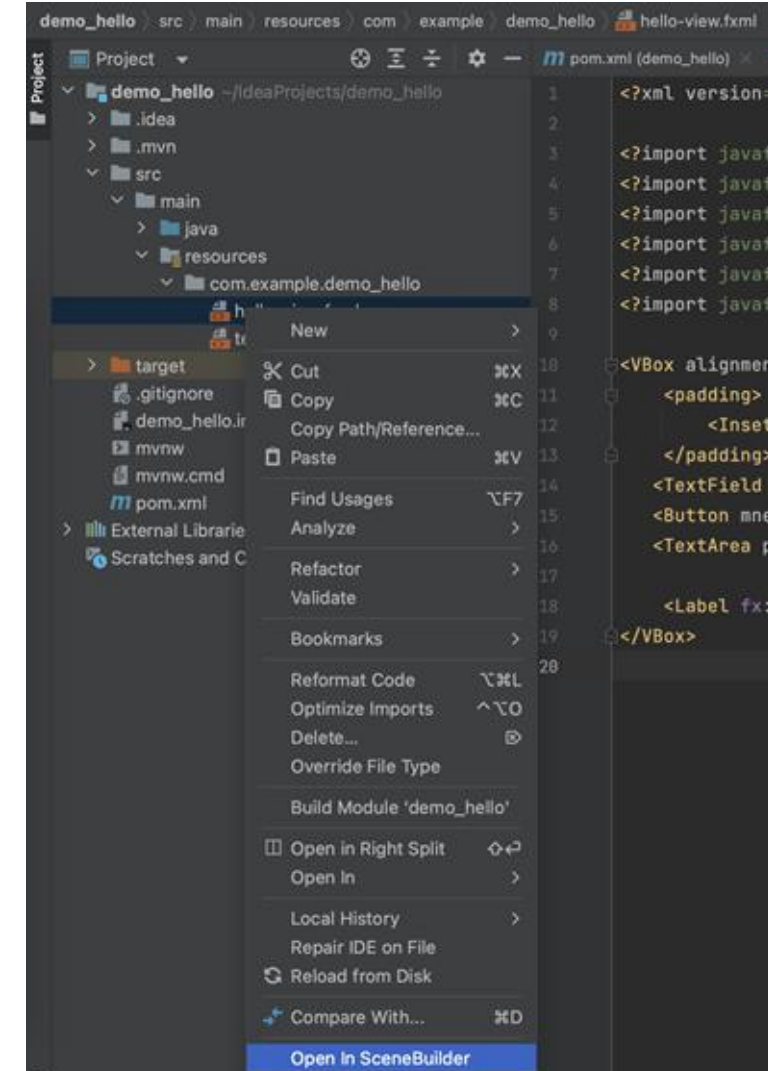
```
public class HelloFontFace extends Application {  
    @Override public void start(Stage primaryStage) {  
        Label label = new Label("Hello @FontFace");  
        label.setStyle("-fx-font-family: sample; -fx-font-size: 80;");  
    }  
}
```

Scene Builder

- Scene Builder là công cụ trực quan, hỗ trợ thao tác kéo thả cho JavaFX. Hỗ trợ tích hợp vào các IDE, code editor như VSCode, Netbeans, IntelliJ IDEA.



<https://gluonhq.com/products/scene-builder/>



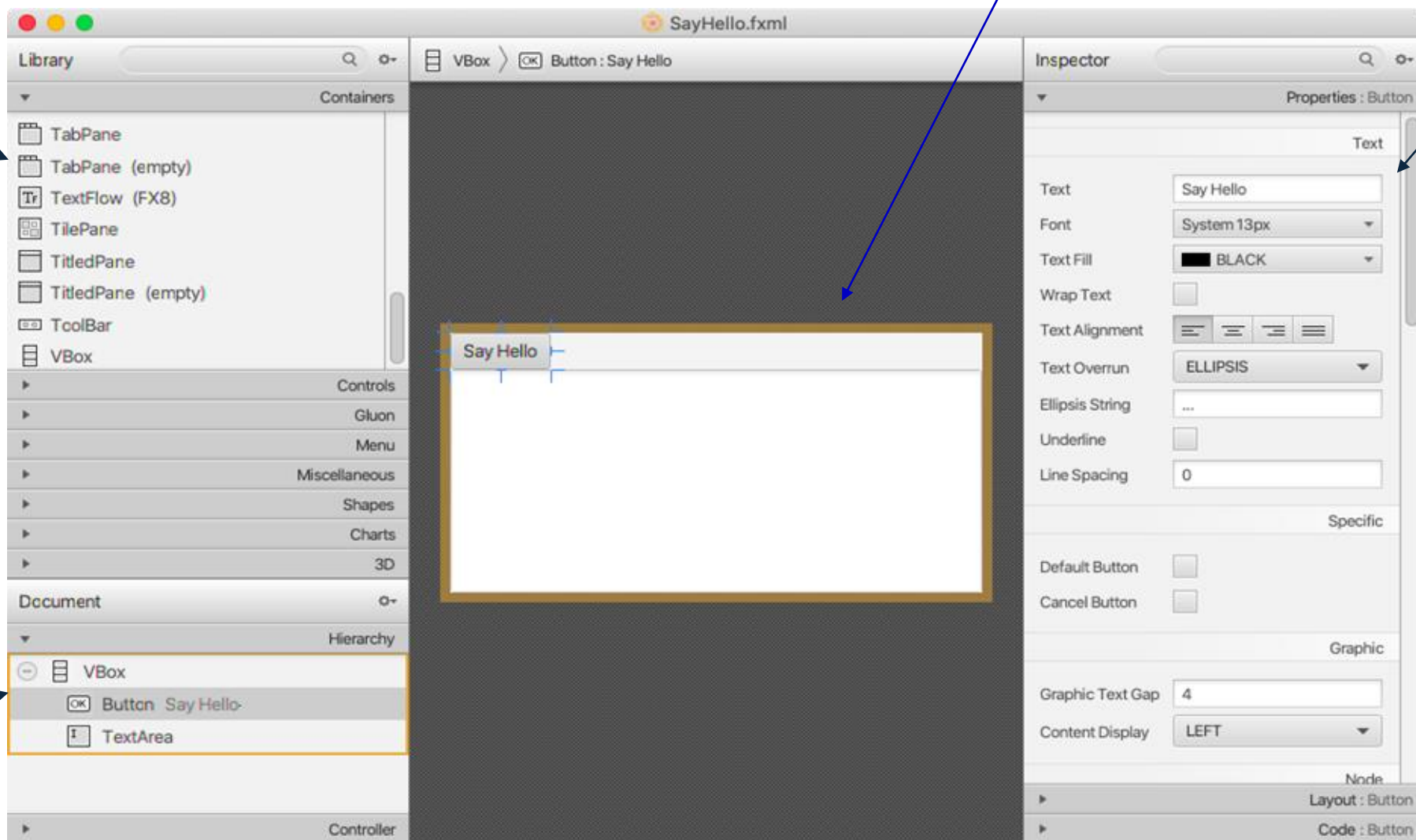
Thao tác với Scene Builder

Các thành phần giao diện

Cấu trúc giao diện

Màn hình xem trước

Các thuộc tính



Xử lý sự kiện trong JavaFX [Controller]

Sự kiện - Event Programing

- **Foreground Events:** người dùng tương tác trực tiếp với *UI controls*
 - Click button
 - Nhập văn bản
 - Chọn item trong list
 - Cuộn trang
- **Background Events:** sự kiện chạy ngầm
 - Báo lỗi
 - Bộ đếm thời gian ngầm

Event Programing

Các loại sự kiện phổ biến bao gồm:

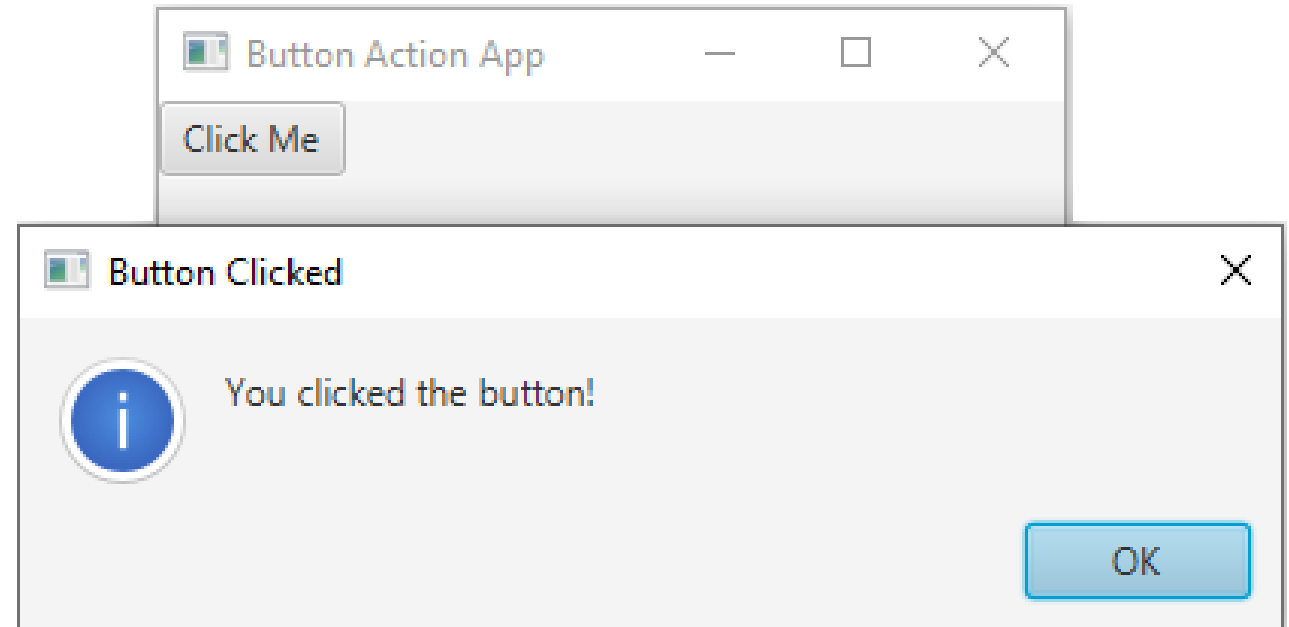
- Mouse Event: click, press, release, move, etc.
- Key Event: press, release.
- Drag and Drop Event.
- Window Event: hide, show.

...

Event Programing

Sự kiện click vào nút:

- Source: chuột
- Target: nút “Click me”
- Type: mouse click



Event Programing

Thiết lập sự kiện trên một đối tượng

`setOn[Event-type](EventHandler<? super [event-class]> value)`

```
playButton.setOnMouseClicked((new EventHandler<MouseEvent>() {  
    public void handle(MouseEvent event) {  
        System.out.println("Hello World");  
        pathTransition.play();  
    }  
}));
```

Event Handlers

Event handlers có thể được sử dụng để *quản lý* các sự kiện cho một đối tượng.

```
//Creating the mouse event handler
EventHandler<javafx.scene.input.MouseEvent> eventHandler =
new EventHandler<javafx.scene.input.MouseEvent>() {
    @Override
    public void handle(javafx.scene.input.MouseEvent e) {
        System.out.println("Hello World");
        circle.setFill(Color.DARKSLATEBLUE);
    }
};

//Adding the event handler
circle.addEventHandler(MouseEvent.MOUSE_CLICKED, eventHandler);

//Removing the event handler
circle.removeEventHandler(MouseEvent.MOUSE_CLICKED, eventHandler);
```

Serialization và lưu trữ dữ liệu [Model]

Làm sao để lưu trữ dữ liệu?

Chuyển đổi dữ liệu dạng text để lưu trữ vào file.



Id 52374

Username example5489@gmail.com

Name example5489

int ID

String email

String name



userInfo.txt

Làm sao để lưu trữ dữ liệu?

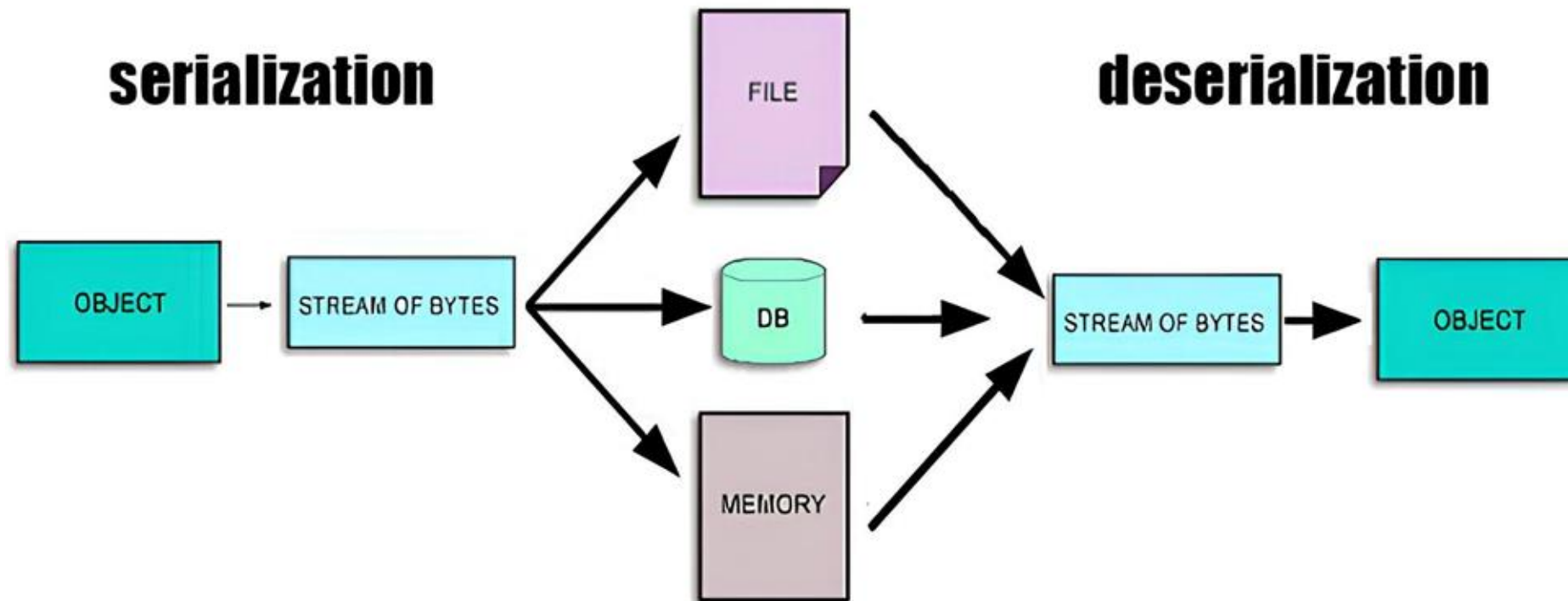
Vậy còn các kiểu dữ liệu phức tạp hơn?

```
public String name;  
public int age;  
public Address address;  
public List<String> courses;
```



Serialization (tuần tự hoá)

Serialization đơn giản là quá trình chuyển đổi đối tượng Java thành dạng dữ liệu có thể truyền hoặc lưu trữ.



Serialization

Lợi ích

- Đọc và ghi một đối tượng thông qua I/O streams.
- Đối tượng cho thể được ghi vào file, truyền qua mạng network hoặc database, v.v.
- Dễ dàng import ngược lại.
- Giảm thiểu sai sót khi chuyển đổi dữ liệu sang dạng văn bản.

Object with data

a: 100	myarray:	(0): 24232
pi: 3.141592		(1): 9823.23
msg: Hello, World!		(2): 12.782

Serialized data

<<100><3.141592><Hello, World!><<24232><9823.23><12.782>>>



Các ứng dụng của Serialization

- Truyền dữ liệu qua mạng
- Lưu trữ trạng thái đối tượng
- Chia sẻ dữ liệu qua các ứng dụng khác nhau
- RMI (Remote Method Invocation)

Triển khai

Lớp cần phải được implement interface `java.io.Serializable`.

```
public class Account implements Serializable {  
    ...  
}
```

serialVersionUID

- Thuộc tính `serialVersionUID` đảm bảo dữ liệu đọc vào và lớp nhận đồng nhất phiên bản và cấu trúc.
- Trình biên dịch có thể tự tính toán `serialVersionUID` nếu nó không được khai báo tường minh.
- Quá trình deserialization có thể ném ngoại lệ `InvalidClassException` nếu hai phiên bản không tương đồng.

```
private static final long serialVersionUID = 1L;
```

Serializable fields

- Để một lớp có thể tuần tự hoá, **tất cả** các thuộc tính trong lớp đó đều phải có thể tuần tự hoá.
- Tất cả dữ liệu nguyên thuỷ đều có thể tuần tự hoá.
- Nhiều kiểu dữ liệu mặc định có thể tuần tự hoá như: `ArrayList`, `HashMap`, `String`, **etc.**

Transient fields

Là thuộc tính tạm thời, sẽ không được tuần tự hoá.

```
private transient type name;
```

Example:

```
private transient String log;
```

Ví dụ: ghi một đối tượng vào file

```
class Account implements Serializable {  
    private static long serialVersionUID = 1L;  
    private int ID;  
    private String name;  
  
    public Account(int ID, String name) {  
        this.ID = ID;  
        this.name = name;  
    }  
}
```

```
Account account = new Account( ID: 2502000, name: "Nguyen Van A");  
  
try (FileOutputStream fos = new FileOutputStream( name: "data.out");  
    ObjectOutputStream output = new ObjectOutputStream(fos);) {  
    output.writeObject(account);  
}  
  
try (FileInputStream fis = new FileInputStream( name: "data.out");  
    ObjectInputStream input = new ObjectInputStream(fis);) {  
    Account accountBackup = (Account) (input.readObject());  
    // Nguyen Van A  
}
```

Kết nối database JDBC & SQLite [Model]

Cơ sở dữ liệu

Cơ sở dữ liệu là một hệ thống các dữ liệu có cấu trúc, nhằm đảm bảo:

- Dữ liệu được tổ chức rõ ràng
- Có thể truy cập đồng thời từ nhiều người dùng, thiết bị

Các hệ quản trị cơ sở dữ liệu phổ biến bao gồm: MySQL, SQL Server, SQLite, MongoDB, v.v

SQLite

Là một hệ quản trị cơ sở dữ liệu có quan hệ với các ưu điểm:

- Gọn nhẹ
- Không cần cấu hình và cài đặt phức tạp
- Dễ dàng lưu trữ vào file
- Tương thích tốt với nhiều thiết bị khác nhau
- ...



SQLite

- Có cấu trúc dạng bảng
- Mỗi bảng bao gồm các trường (id, name, address, v.v)
- Mỗi bảng chứa các bản ghi có các thông tin của các trường.

USER		
id	name	address
1	Nguyễn Văn A	Hanoi
2	Trần Thị B	Hanoi
3	Lê Văn C	Nam Dinh

PRODUCT	
id	name
1	Xà phòng
2	Dầu gội
3	Nước mắt

Ví dụ: SQLite

Tạo bảng

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    password TEXT  
);
```

Khoá chính

Tự động tăng

Tên trường

Kiểu dữ liệu

Chèn dữ liệu

```
INSERT INTO users (name, email, password) VALUES ('John Doe',  
'john.doe@example.com', 'password123');
```

Truy vấn dữ liệu

```
SELECT name, email FROM users WHERE id = 1;
```

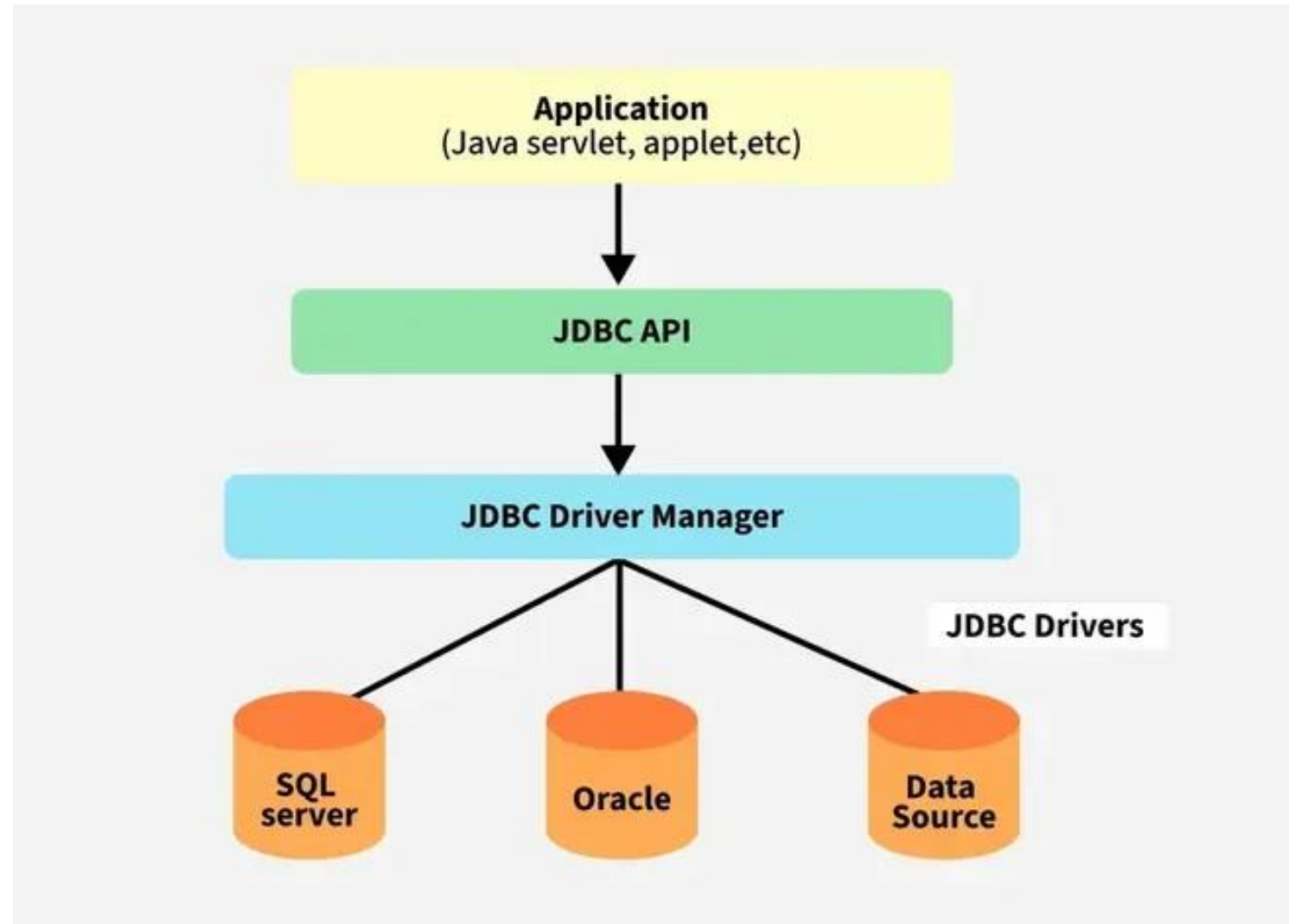
Cập nhật dữ liệu

```
UPDATE users SET email = 'john.doe@newdomain.com' WHERE id = 1;
```

- JDBC (Java Database Connectivity) là một API (Application Programming Interface - Giao diện Lập trình Ứng dụng) chuẩn của Java để kết nối các cơ sở dữ liệu.
 - JDBC API: cho phép ứng dụng kết nối đến JDBC Manager connection.
 - JDBC Driver API: Hỗ trợ JDBC Manager đến Driver connection.

=> Cho phép ứng dụng có thể kết nối đến các DB khác nhau một cách đồng nhất

JDBC



Ví dụ: Kết nối JDBC và SQLite

Bước 1. Tạo cơ sở dữ liệu dạng file
(trên CMD, Terminal)

sqlite3 data.db

Bước 2. Tạo kết nối

```
Connection conn = DriverManager.getConnection(url)
String url = "jdbc:sqlite:data.db";
String url = "jdbc:sqlite:data.db";
```

Bước 3. Tạo bảng

```
String sql = "CREATE TABLE IF NOT EXISTS users ("
    + " id INTEGER PRIMARY KEY,"
    + " name TEXT NOT NULL,"
    + " address TEXT,"
    + " phone_number TEXT"
    + ");";

try (Connection conn = DriverManager.getConnection(url);
    Statement stmt = conn.prepareStatement(sql)) {
    if (conn != null) {
        stmt.execute(sql);
    }
} catch (Exception e) {
    e.printStackTrace();
}
```

Ví dụ: Kết nối JDBC và SQLite

- url: jdbc:<driver>:<file db>
- preparedStatement() : biên dịch trước để tối ưu và đảm bảo bảo mật cho câu lệnh.
- execute(sql) : thực thi câu lệnh.

```
String url = "jdbc:sqlite:data.db";

String sql = "CREATE TABLE IF NOT EXISTS users ("
    + " id INTEGER PRIMARY KEY,"
    + " name TEXT NOT NULL,"
    + " address TEXT,"
    + " phone_number TEXT"
    + ");";

try (Connection conn = DriverManager.getConnection(url);
    Statement stmt = conn.prepareStatement(sql)) {
    if (conn != null) {
        stmt.execute(sql);
    }
} catch (Exception e) {
    e.printStackTrace();
}
```

Ví dụ: Kết nối JDBC và SQLite

Sử dụng phương thức `prepareStatement()`

```
String sql = "INSERT INTO students (id, name, address) VALUES (?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1, id);
pstmt.setString(2, name);
pstmt.setString(3, address);
pstmt.executeUpdate();
```

Ví dụ: mẫu thiết kế Singleton

- Là mẫu thiết kế khởi tạo (Creational)
- Đảm bảo 1 class chỉ có thể có duy nhất 1 instance
- Sử dụng một constructor private
- Chỉ duy nhất một đối tượng làm nhiệm vụ kết nối tới database trong một thời điểm

=> Tiết kiệm tài nguyên, dễ dàng quản lý

```
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;
    private final String url = "jdbc:sqlite:data.db";

    private DatabaseConnection() throws SQLException {
        try {
            Class.forName("org.sqlite.JDBC");
            this.connection = DriverManager.getConnection(url);
        } catch (ClassNotFoundException e) {
            throw new SQLException("SQLite Driver not found", e);
        }
    }

    public Connection getConnection() {
        return connection;
    }

    public static synchronized DatabaseConnection getInstance() throws SQLException {
        if (instance == null || instance.getConnection().isClosed()) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}
```

Ví dụ: Serialization và JBDC

Mục tiêu: Lưu trữ các kiểu phức hợp trong database. Sử dụng kiểu dữ liệu BLOB (Binary Large Object)

```
CREATE TABLE IF NOT EXISTS data_saves (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL,  
    save_data BLOB NOT NULL,  
);
```

```
String sql = "INSERT INTO data_saves(id, save_data) VALUES(?, ?)";  
  
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {  
    pstmt.setString(1, id);  
  
    // Chuyển Object thành byte array  
    ByteArrayOutputStream baos = new ByteArrayOutputStream();  
    ObjectOutputStream oos = new ObjectOutputStream(baos);  
    oos.writeObject(data);  
    byte[] bytes = baos.toByteArray();  
  
    pstmt.setBytes(2, bytes); // Đẩy mảng byte vào cột BLOB  
    pstmt.executeUpdate();  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

Ví dụ: JSON Serialization

- JSON: JavaScript Object Notation
- Vẫn lưu trữ trong DB dưới dạng **text** đơn thuần
- Không bị ảnh hưởng bởi định nghĩa class và serialVersionUID
- Hỗ trợ nhiều ngôn ngữ khác nhau

Ví dụ: JSON Serialization

Bước 1. Thêm thư viện vào Maven

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.10.1</version>
</dependency>
```

Bước 2. Định nghĩa lớp (tương tự với Native Serialization)

```
class BagOfPrimitives {
    private int value1 = 1;
    private String value2 = "abc";
    private transient int value3 = 3;
    BagOfPrimitives() {
        // no-args constructor
    }
}
```

```
// Serialization
BagOfPrimitives obj = new BagOfPrimitives();
Gson gson = new Gson();
String json = gson.toJson(obj);
```

Dạng text thuần túy

```
// Deserialization
BagOfPrimitives obj2 = gson.fromJson(json, BagOfPrimitives.class);
// ==> obj2 is just like obj
```

Thực hành

- Xây dựng ứng dụng JavaFX với mô hình MVC
- Kết nối đến cơ sở dữ liệu với JDBC